

A Physics-Informed Neural Operator for Heterogeneous Contaminant Transport in a Four-Zone Aquifer

Ali Haghighi

Supervised by:

Afshin Ashrafzadeh, François Lehmann, Marwan Fahs

June 12, 2026



Abstract

Reports 1–4 developed physics-informed surrogates that reproduce one-dimensional *homogeneous* advection–diffusion transport to near-perfect accuracy [1]. This report extends the DeepONet-style physics-informed neural operator (PINO) to a *heterogeneous*, four-zone medium (zone lengths 20–20–20–40 m) with a separate velocity per zone, learning the map from the four zone velocities to the concentration field $C^*(x^*, t^*)$. We evaluate the operator against 81 COMSOL reference solutions and compare a small set of training choices: how the training media are sampled (Latin-hypercube, with and without maximin spacing), the floating-point precision, and the L-BFGS line search. Maximin sampling was more accurate than plain Latin-hypercube, and the largest reductions came from the optimizer settings, taking the error against COMSOL—a relative L_2 measure (defined in Section 3)—from roughly 10–13% down to a few percent. We describe the operator, the training setup, the experiments, and the remaining limitation—a dependence on the fixed four-zone geometry—which motivates the next step toward a function-input operator.

1 Introduction

Over the first four reports this project developed physics-informed surrogates for one-dimensional contaminant transport and brought them to near-perfect accuracy on *homogeneous* media. Report 1 introduced a baseline PINN; Report 2 stabilized it with Xavier initialization and added a parametric PINN over the Péclet number; Report 3 documented the baseline PINN, parametric PINN, and a DeepONet-style operator, all near-perfect; and Report 4 placed the three surrogates in one clean test family (fixed diffusion, with the advection varied) and matched COMSOL to under 1% in every case [1].

The natural next step is a *heterogeneous* medium. We adopt a stratified aquifer of four zones, each with its own pore velocity, and ask the operator to learn

$$(u_1, u_2, u_3, u_4) \longmapsto C^*(x^*, t^*),$$

generalizing across per-zone velocities of 0.01, 0.03, and 0.05 m d^{−1} (the three levels used in the COMSOL reference) and the continuous range between them. The goal of this report is a single operator that is accurate for any combination of the four zone velocities on this fixed geometry, trained as efficiently as possible.

A first operator, trained exactly as in Report 4, reached only about 10% error against the COMSOL cases and did not improve with longer training or with more training media. Even *homogeneous* cases (all four velocities equal)—which are the same problem Report 4 solves to under 1%—were off by roughly 8%. Because the difficulty was not confined to the heterogeneity, we ran a set of controlled experiments varying how the training media are sampled, the numerical precision, and the L-BFGS settings, and report the comparison below.

Table 1: Physical setup and dimensionless groups (as in Report 4).

Quantity	Value
Domain length L	100 m (interfaces at 20, 40, 60 m)
Time horizon $T_{\max}$	1200 d
Inlet concentration C_0	5 kg m ^{−3}
Diffusion D	10 ^{−6} m ² s ^{−1}
Zone velocities u_i	[0.01, 0.05] m d ^{−1}
Per-zone CFL _{i} = $u_i T_{\max}/L$	[0.12, 0.60]
Fixed Pe = $DT_{\max}/L^2$	0.0104

2 Problem Setup

We model one-dimensional transport with a piecewise-constant velocity $U(x)$ and constant diffusion D ,

$$\frac{\partial C}{\partial t} + U(x) \frac{\partial C}{\partial x} = D \frac{\partial^2 C}{\partial x^2}, \quad (1)$$

with $C(x, 0) = 0$, $C(0, t) = C_0$, $C(L, t) = 0$. The domain $[0, L]$ is split into four zones with the COMSOL lengths, each carrying a velocity u_i :

$$\underbrace{[0, 20]}_{u_1} \underbrace{[20, 40]}_{u_2} \underbrace{[40, 60]}_{u_3} \underbrace{[60, 100]}_{u_4} \text{ m.}$$

Diffusion and the time horizon are fixed exactly as in Report 4 (Table 1); only the four velocities vary, each over $[0.01, 0.05]$ m d^{−1}.

Using $x^* = x/L$, $t^* = t/T_{\max}$, $C^* = C/C_0$, Eq. (1) becomes

$$\frac{\partial C^*}{\partial t^*} + \text{CFL}(x^*) \frac{\partial C^*}{\partial x^*} - \text{Pe} \frac{\partial^2 C^*}{\partial x^{*2}} = 0, \quad (2)$$

where $\text{CFL}(x^*) = U(x) T_{\max}/L$ is the dimensionless advection rate (the Courant–Friedrichs–Lewy number; written CFL hereafter) and $\text{Pe} = DT_{\max}/L^2$ is the fixed diffusive group. Because U is piecewise constant, $\text{CFL}(x^*)$ equals CFL_i on zone i and jumps at $x^* = 0.2, 0.4, 0.6$. Conditions become $C^*(x^*, 0) = 0$, $C^*(0, t^*) = 1$, $C^*(1, t^*) = 0$. The reference set is 81 COMSOL solutions on the velocity grid $\{0.01, 0.03, 0.05\}^4$, used only for evaluation; they never enter any training loss.

3 Operator and Training

Architecture. We use a DeepONet-style operator [2] with two networks. A *branch* network maps the four zone values ($\text{CFL}_1, \dots, \text{CFL}_4$) through hidden layers of width 16, 16, 32; a *trunk* network maps the coordinates (x^*, t^*) through four hidden layers of width 32. Both use Tanh activations with Xavier initialization [3] and output a length-32 feature vector (b_j from the branch, τ_j from the trunk). The prediction is their inner product passed through a logistic sigmoid σ , which keeps the concentration in $[0, 1]$:

$$\hat{C}^*(x^*, t^*; \mathbf{CFL}) = \sigma\left(\sum_{j=1}^{32} b_j(\mathbf{CFL}) \tau_j(x^*, t^*)\right). \quad (3)$$

The same network is used in every experiment below.

Physics-informed loss. Training uses physics only [4]; the loss is the equally weighted sum of the residual of Eq. (2) and the initial, inlet, and outlet conditions, each a mean-squared error. Although the branch receives all four zone values, the

residual at a collocation point uses only the local one: the point’s coordinate x^* determines which zone it lies in, and the matching CFL_i is selected for that point (a “where” operation on x^*). The four inputs are therefore a per-zone library from which each point takes its own coefficient.

Optimizer. As in Report 4, training uses full-batch L-BFGS. We vary three settings between experiments—the numerical precision (`float32` or `float64`), the L-BFGS step size, and the number of line-search iterations per step—and hold everything else fixed.

Collocation. For each training medium the residual is enforced on a fixed structured mesh in (x^*, t^*) (Fig. 1). The interior set combines a 50×50 bulk grid with a refinement band at each zone interface—12 extra x^* -lines within ± 0.025 of $x^* = 0.2, 0.4, 0.6$, where the solution slope changes most sharply. This gives 86 distinct x^* positions (50 bulk + 36 interface) at each of 50 time levels, i.e. 4,300 interior points per medium; the initial, inlet, and outlet conditions add 50 points each. The full training set is the union of these over all media, so it grows with the number of media—about 2.2 million points for the 500-medium runs, all evaluated at every L-BFGS step (full batch).

Accuracy measure. We compare the operator with the COMSOL reference through a relative L_2 error. For one medium it is the spatial L_2 norm of $(\hat{C}^* - C_{\text{COMSOL}}^*)$ divided by the norm of C_{COMSOL}^* , evaluated at the five COMSOL output times and averaged. The mean, maximum, and minimum quoted below are this quantity taken over the 81 reference media.

4 Experiments

We hold the network and the COMSOL evaluation fixed and compare a few training choices. Each training medium is one draw of the four zone velocities, i.e. a single point in the four-dimensional hypercube $[0.01, 0.05]^4$, and we need a set of N such media that covers this hypercube well. We draw them by four-dimensional Latin-hypercube (LHC) sampling [5]: each of the four velocity axes is split into N equal bins, and the N media are placed so that every bin of every axis is used exactly once. This guarantees that each velocity *individually* is evenly spread over its range, but it constrains only the one-dimensional margins—many such designs still leave the media clumped or aligned in the joint four-dimensional space. We therefore test two variants: *plain* LHC, a single random such design, and *maximin* LHC, which searches over LHC designs for the one that maximizes the smallest pairwise distance between media, pushing them apart so they fill the four-dimensional hypercube more uniformly [6]. We also vary the numerical precision and the L-BFGS line search.

The five runs in Table 2 are selected from a larger set of exploratory experiments (which also varied the network size and other settings) as the ones that isolate the trends; the remainder were intermediate or redundant. Their roles are:

- **A** trains on the 81 reference media themselves (in-sample), testing whether the operator can fit these solutions at all when it sees them during training.
- **B** trains on 500 plain-LHC media, testing generalization from a basic space-filling design.
- **C** trains on 500 maximin-LHC media, testing whether better-spread media help.
- **I** repeats run C in `float64` with a reduced L-BFGS step.
- **J** repeats run I with more line-search iterations per L-BFGS step.

Runs A, B, C use `float32`; I and J use `float64`; all share the same network.

5 Results

Comparison of training choices. Table 2 and Fig. 2 summarize the runs. Among the `float32` runs the error stays between about 10% and 14%: training in-sample (A, 13.6%) and on plain-LHC media (B, 13.4%) are close, while maximin sampling (C, 10.3%) is the most accurate of the three. Keeping that same network and the same maximin media, the two `float64` runs reach 6.4% (I) and 2.6% (J). The homogeneous cases (all four velocities equal) follow the same pattern—about 8% in `float32` and improving with the `float64` runs—even though they contain no interface and match the problem Report 4 solved to under 1%. Taken together, sampling design moved the error by a few points and maximin was clearly the best sampler, while the largest changes we observed came from the precision and line-search settings.

Profiles and residuals. Figure 3 compares run C (`float32`, the best sampler) and run J (`float64`, tuned) against COMSOL on three media: a homogeneous field, an alternating high-contrast field with a jump at every interface, and a two-step field. Run C lags the reference, with the front arriving too early; run J overlaps COMSOL across all times, including the high-contrast case. As expected for a piecewise coefficient, the PDE residual concentrates at the zone interfaces; in run J this does not translate into a large solution error, and the interfaces are reproduced with a single shared trunk. Whether a per-zone (domain-decomposition) architecture would help beyond this accuracy was not tested here.

Where the residual sits. In run J the remaining error is largest for the media with the slowest inlet zone and is concentrated near the inlet ($x^* = 0$) rather than at the interior interfaces.

6 Conclusions and Next Steps

On this four-zone problem we compared how the operator’s accuracy depends on the training media, the numerical precision, and the L-BFGS settings. Maximin Latin-hypercube sampling was the most accurate sampler, and the largest reductions in error came from the precision and line-search settings, taking the tuned operator (run J) to within a few percent of COMSOL on all 81 media using a single shared trunk. The

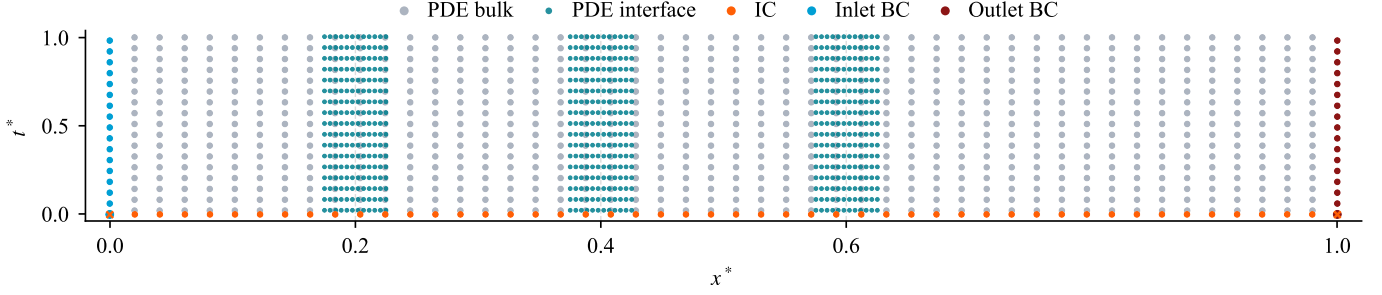


Figure 1: Training collocation in the (x^*, t^*) plane for one representative medium, $u = (0.03, 0.03, 0.03, 0.03)$. PDE *bulk* points lie on a 50×50 grid (grey); extra PDE *interface* points refine the bands within ± 0.025 of $x^* = 0.2, 0.4, 0.6$ (teal); the initial condition at $t^* = 0$ (orange) and the inlet/outlet boundaries at $x^* = 0, 1$ (blue/dark red) complete the set. The layout is *schematic*—points are thinned and the axes and marker sizes are cosmetic—so it conveys the mesh structure rather than the literal count. Per medium the mesh has 86 distinct x^* positions $\times$ 50 time levels (4,300 interior points), plus 50 points each for the IC, inlet, and outlet.

Table 2: The five runs, all using the same network and evaluated on the 81 COMSOL media. “Mean/Max/Min” are the relative L_2 error (%) defined in Section 3. “L-BFGS step” is the step size and “line searches” the number of line-search iterations per L-BFGS step. Runs A–C differ only in the training media; I and J change only the precision and the L-BFGS settings, on the same maximin media as C.

Run	Training media	Purpose	Precision	L-BFGS step / line searches	Mean	Max	Min
A	81 reference (in-sample)	can it fit at all?	float32	1 / 1	13.6	21.9	9.6
B	500 plain LHC	generalize from plain sampling	float32	1 / 1	13.4	21.3	8.1
C	500 maximin LHC	does better spacing help?	float32	1 / 1	10.3	20.3	5.9
I	500 maximin LHC	effect of precision & step	float64	0.1 / 1	6.4	16.1	3.2
J	500 maximin LHC	effect of deeper line search	float64	0.1 / 20	2.6	4.9	1.6

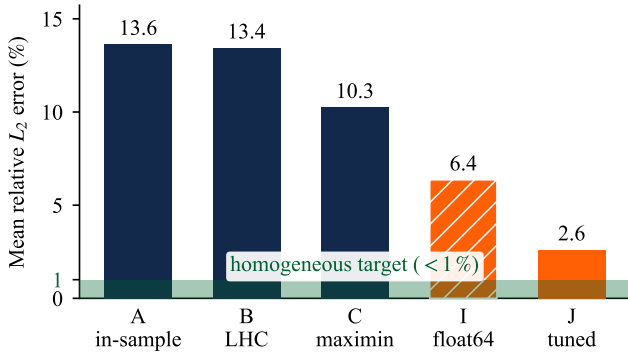


Figure 2: Mean relative L_2 error over the 81 COMSOL media for the five runs. Navy: float32 runs differing only in the training media (in-sample A, plain LHC B, maximin C). Orange: float64 runs on the same maximin media, with a reduced L-BFGS step (hatched, I) and additionally more line-search iterations (J). The green band marks the $< 1\%$ accuracy reached on homogeneous media in Report 4.

homogeneous cases improved in step with the heterogeneous ones, so the difficulty was not specific to the zone structure.

The main limitation of the present design is that it is tied to the fixed four-zone geometry: the branch takes exactly four scalars, so it cannot transfer to a different number or arrangement of zones. The next step is to replace the four-scalar input with a velocity *field* sampled at fixed sensor locations and to train over Gaussian-random-field draws of that field, which would make the operator independent of the zonation and applicable to smooth and layered media alike. That is the subject of the next report.

References

- [1] Ali Haghighi et al. *Reports 1–4: Physics-Informed Neural Networks and Operators for One-Dimensional Homogeneous Contaminant Transport*. 2025–2026. URL: <https://github.com/Aliaox/pinn/blob/main/docs/reports> (visited on 06/10/2026).
- [2] Lu Lu et al. “Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators”. In: *Nature Machine Intelligence* 3.3 (Mar. 2021), pp. 218–229. ISSN: 2522-5839. DOI: [10.1038/s42256-021-00302-5](https://doi.org/10.1038/s42256-021-00302-5). arXiv: [1910.03193](https://arxiv.org/abs/1910.03193). URL: <http://dx.doi.org/10.1038/s42256-021-00302-5>.
- [3] Xavier Glorot and Yoshua Bengio. “Understanding the difficulty of training deep feedforward neural networks”. In: *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*. Vol. 9. Proceedings of Machine Learning Research. JMLR Workshop and Conference Proceedings, 2010, pp. 249–256. URL: <https://proceedings.mlr.press/v9/glorot10a/glorot10a.pdf>.
- [4] Maziar Raissi, Paris Perdikaris, and George Em Karniadakis. “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations”. In: *Journal of Computational Physics* 378 (2019), pp. 686–707. DOI: [10.1016/j.jcp.2018.10.045](https://doi.org/10.1016/j.jcp.2018.10.045).
- [5] Michael D. McKay, Richard J. Beckman, and William J. Conover. “A comparison of three methods for selecting values of input variables in the analysis of output from a

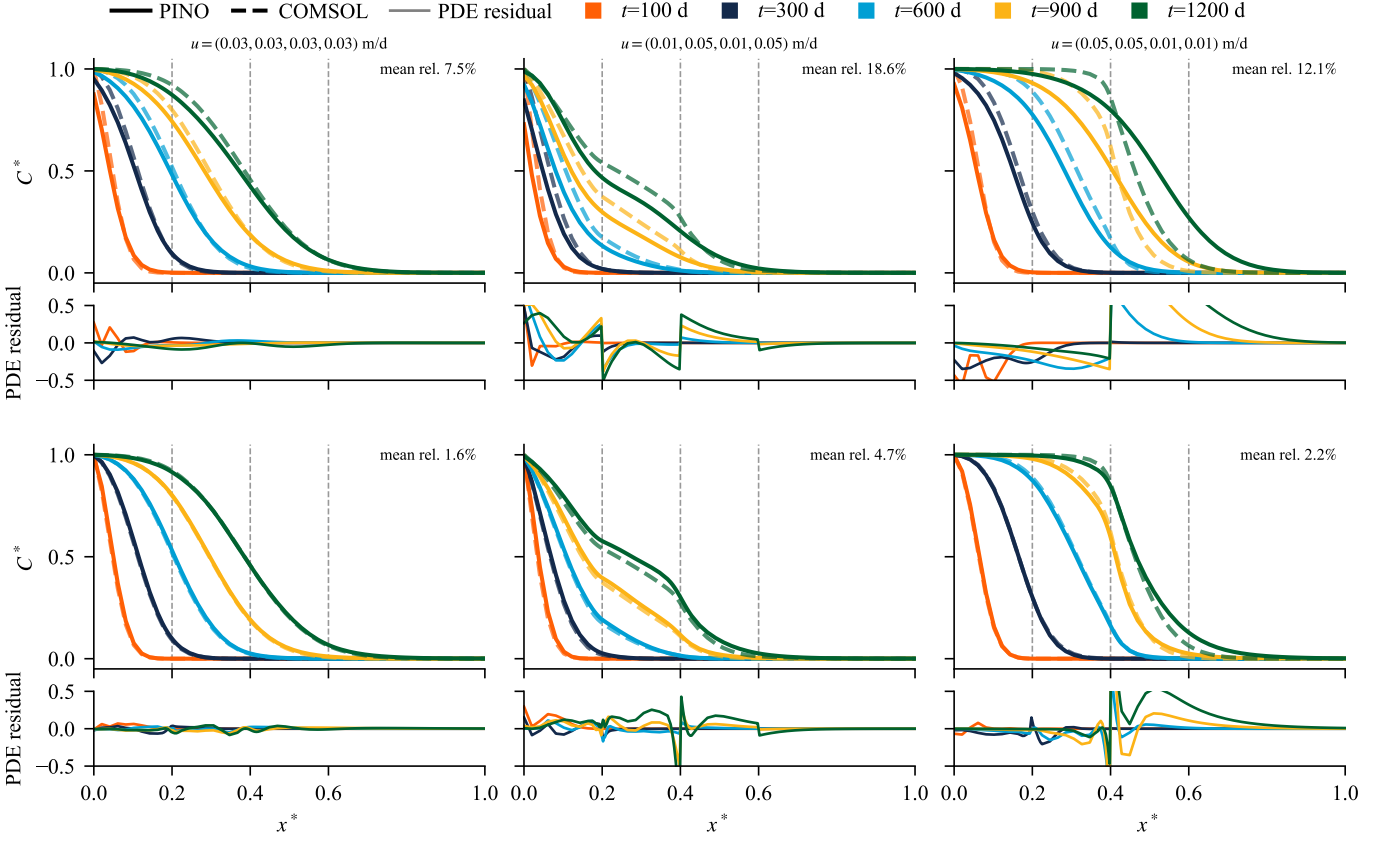


Figure 3: Validation against COMSOL on three media (columns: homogeneous, alternating high-contrast, and two-step; velocities in the headers). **Top block: run C (float32); bottom block: run J (float64, tuned).** In each block the upper panel shows the concentration C^* (PINO solid, COMSOL dashed) at five times, with the per-panel mean relative L_2 error annotated, and the lower strip shows the PDE residual; vertical dashed lines mark the interfaces $x^* = 0.2, 0.4, 0.6$. Run C’s front arrives early (worst on the alternating case, 18.6%), whereas run J overlaps COMSOL throughout ($\leq 4.7\%$)—the residual still peaks at the interfaces, but the solution stays accurate.

computer code”. In: *Technometrics* 21.2 (1979), pp. 239–245. DOI: [10.1080/00401706.1979.10489755](https://doi.org/10.1080/00401706.1979.10489755).

- [6] Max D. Morris and Toby J. Mitchell. “Exploratory designs for computational experiments”. In: *Journal of Statistical Planning and Inference* 43.3 (1995), pp. 381–402. DOI: [10.1016/0378-3758\(94\)00035-T](https://doi.org/10.1016/0378-3758(94)00035-T).